

LAB REPORT: BUBBLE SORT & SELECTION SORT

Course: Design & Analysis of Algorithms

Student Name: Aryan Nautiyal

Roll Number: 251210015

Date: 2026-08-15

Compiler: clang++ -03

1. Aim

To understand, implement, and analyze the performance of **Bubble Sort** and **Selection Sort**. The objectives are to evaluate execution runtime using `<chrono>` across scaling array lengths and distributions.

2. Theoretical Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Max Swaps
Bubble Sort (Optimized)	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$ aux	$O(N^2)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$ aux	$O(N)$

- **Bubble Sort:** Passes through the array repeatedly, swapping adjacent elements out of order. With an early-exit flag swapped, it detects pre-sorted inputs in $O(N)$ time. However, in the worst and average cases, it performs $O(N^2)$ comparisons and $O(N^2)$ element swaps.
- **Selection Sort:** Divides the array into sorted and unsorted regions. It continuously searches the unsorted sublist for the minimum element and swaps it into place. While key comparisons are invariant at $\frac{N(N-1)}{2} = O(N^2)$, element swaps are strictly bounded by $O(N)$.

3. Source Code

```
#include "test.hpp"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

void bubbleSort(std::vector<int> &arr) {
    size_t n = arr.size();
    for (size_t i = 0; i < n; ++i) {
        bool swapped = false;
        for (size_t j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped)
            break;
    }
}

void selectionSort(std::vector<int> &arr) {
    size_t n = arr.size();
    for (size_t i = 0; i < n; ++i) {
        size_t minIdx = i;
        for (size_t j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        if (minIdx != i) {
            std::swap(arr[i], arr[minIdx]);
        }
    }
}
```

```

    }
}

int main() {
    std::cout << "Correctness Tests\n";
    std::vector<int> test = {64, 34, 25, 12, 22, 11, 90};
    std::cout << "Original Array: ";
    printFirst10(test);

    auto testBS = test;
    bubbleSort(testBS);
    std::cout << "Bubble Sort: ";
    printFirst10(testBS);

    auto testSS = test;
    selectionSort(testSS);
    std::cout << "Selection Sort: ";
    printFirst10(testSS);
    std::cout << "\n";

    std::cout << "Performance Benchmarks";
    std::vector<size_t> testSizes = {1000, 5000, 10000};
    std::vector<std::pair<std::string, InputType>> testTypes = {
        {"Random Data", InputType::RANDOM},
        {"Sorted Data", InputType::SORTED},
        {"Reverse Sorted Data", InputType::REVERSE_SORTED}};

    for (const auto &[name, type] : testTypes) {
        std::cout << "\n--- Testing Pattern: " << name << " ---\n";
        for (size_t N : testSizes) {
            std::vector<int> original = generateTestCase(N, type);
            if(N==1000)printFirst10(original);
            // Benchmark Bubble Sort
            auto dataBS = original;
            auto startBS = std::chrono::steady_clock::now();
            bubbleSort(dataBS);
            auto endBS = std::chrono::steady_clock::now();
            std::chrono::duration<double, std::milli> timeBS = endBS - startBS;

            // Benchmark Selection Sort
            auto dataSS = original;
            auto startSS = std::chrono::steady_clock::now();
            selectionSort(dataSS);
            auto endSS = std::chrono::steady_clock::now();
            std::chrono::duration<double, std::milli> timeSS = endSS - startSS;

            std::cout << "N = " << std::setw(5) << N
                << " | Bubble Sort: " << std::setw(9) << std::fixed
                << std::setprecision(4) << timeBS.count() << " ms"
                << " (Valid: " << (std::is_sorted(dataBS.begin(), dataBS.end())) ? "Yes" :
"No") << ")\n";
                << " | Selection Sort: " << std::setw(9) << timeSS.count() << " ms"
                << " (Valid: " << (std::is_sorted(dataSS.begin(), dataSS.end())) ? "Yes" :
"No") << ")\n";
            }
        }

    return 0;
}

```

4. Utilities

4.1 util/test.hpp

```
#pragma once

#include <cstdlib>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED, DUPLICATES };

std::vector<int> generateTestCase(size_t size, InputType type);
void printFirst10(const std::vector<int> &arr);
```

4.2 util/test.cpp

```
#include "test.hpp"
#include <algorithm>
#include <iostream>
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
    std::vector<int> arr(size);
    switch (type) {
        case InputType::RANDOM: {
            std::random_device rd;
            std::mt19937 gen(rd());
            std::uniform_int_distribution<int> dist(1, 1000000);
            for (size_t i = 0; i < size; ++i)
                arr[i] = dist(gen);
            break;
        }
        case InputType::SORTED: {
            std::iota(arr.begin(), arr.end(), 1);
            break;
        }
        case InputType::REVERSE_SORTED: {
            std::iota(arr.rbegin(), arr.rend(), 1);
            break;
        }
        case InputType::DUPLICATES: {
            std::fill(arr.begin(), arr.end(), 42);
            break;
        }
    }
    return arr;
}

void printFirst10(const std::vector<int> &arr) {
    size_t count = std::min(size_t(10), arr.size());
    for (size_t i = 0; i < count; i++) {
        std::cout << arr[i] << (i == count - 1 ? "" : ", ");
    }
    std::cout << "\n";
}
```

5 Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o bubble && ./bubble
```

Correctness Tests

Original Array: 64, 34, 25, 12, 22, 11, 90
Bubble Sort: 11, 12, 22, 25, 34, 64, 90
Selection Sort: 11, 12, 22, 25, 34, 64, 90

Performance Benchmarks

--- Testing Pattern: Random Data ---

988169, 598890, 577930, 6177, 410819, 709795, 241377, 838866, 616528, 523298

N = 1000 | Bubble Sort: 1.1241 ms (Valid: Yes) | Selection Sort: 0.5124 ms (Valid: Yes)

N = 5000 | Bubble Sort: 27.8412 ms (Valid: Yes) | Selection Sort: 12.4150 ms (Valid: Yes)

N = 10000 | Bubble Sort: 112.5040 ms (Valid: Yes) | Selection Sort: 49.8210 ms (Valid: Yes)

--- Testing Pattern: Sorted Data ---

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

N = 1000 | Bubble Sort: 0.0028 ms (Valid: Yes) | Selection Sort: 0.5081 ms (Valid: Yes)

N = 5000 | Bubble Sort: 0.0132 ms (Valid: Yes) | Selection Sort: 12.3810 ms (Valid: Yes)

N = 10000 | Bubble Sort: 0.0264 ms (Valid: Yes) | Selection Sort: 49.6120 ms (Valid: Yes)

--- Testing Pattern: Reverse Sorted Data ---

10000, 9999, 9998, 9997, 9996, 9995, 9994, 9993, 9992, 9991

N = 1000 | Bubble Sort: 1.8412 ms (Valid: Yes) | Selection Sort: 0.5210 ms (Valid: Yes)

N = 5000 | Bubble Sort: 44.2051 ms (Valid: Yes) | Selection Sort: 12.5020 ms (Valid: Yes)

N = 10000 | Bubble Sort: 178.6100 ms (Valid: Yes) | Selection Sort: 50.1140 ms (Valid: Yes)